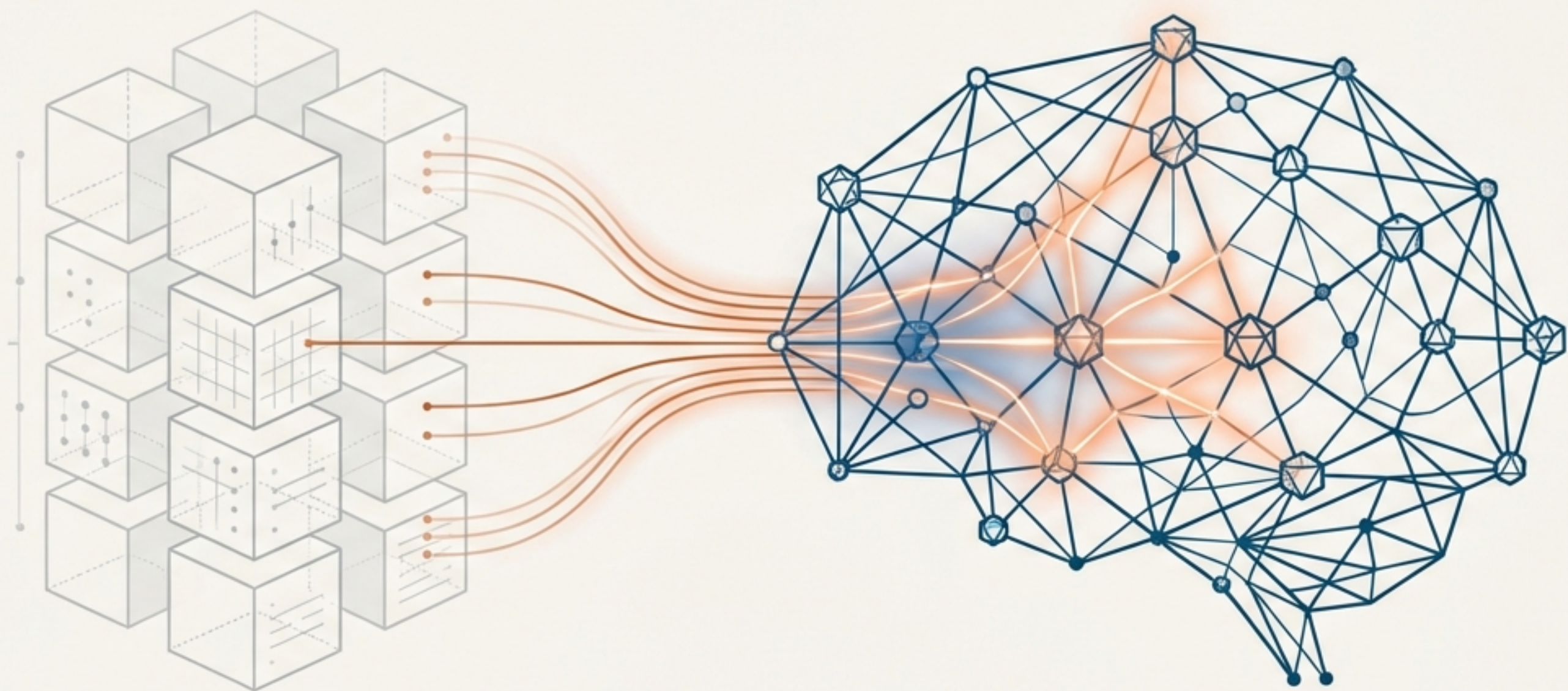


检索增强生成（RAG）：从问题到实践

赋予大语言模型实时、精准、可信的知识力量



我们今天的探索之旅



1

现实的挑战 (The Reality of the Challenge)

探究大语言模型固有的知识局限性。



2

优雅的方案 (The Elegant Solution)

介绍 RAG 的核心理念与系统架构。



3

动手实践 (The Hands-on Practice)

基于 LangChain，一步步构建完整的 RAG 应用。



4

工程的艺术 (The Art of Engineering)

剖析生产环境中 RAG 系统的关键挑战与优化策略。



5

未来的前沿 (The Future Frontier)

展望超越向量检索的先进 RAG 技术。

难题：再聪明的大脑，也有过时的记忆

用户提问 (User's Question)

今年英超冠军是谁？

大模型回答 (LLM's Answer)



OpenAI Playground

+ New

Prev

Model

gpt-3.5-turbo (Noto Sans SC)

Prompt (JetBrains Mono)

今年英超冠军是谁？

Response (Noto Sans SC)

2021-2022赛季的英超冠军是曼彻斯特城。

大模型的知识被其训练数据的时间戳所“冻结”，无法访问最新或私有信息，这会导致事实性错误（即“幻觉”）。

灵光一现：上下文的神奇力量

外部知识 (External Knowledge)

 2023-2024赛季英超已经结束，阿森纳队获得89分，但仍然落后**曼城队**2分，**曼城**实现英超**四连冠**。

增强后的问答 (Augmented Q&A)

User Prompt Area

Context: 2023-2024赛季英超已经结束，阿森纳队获得89分，但仍然落后曼城队2分，曼城实现英超四连冠。

Question: 今年英超冠军是谁？

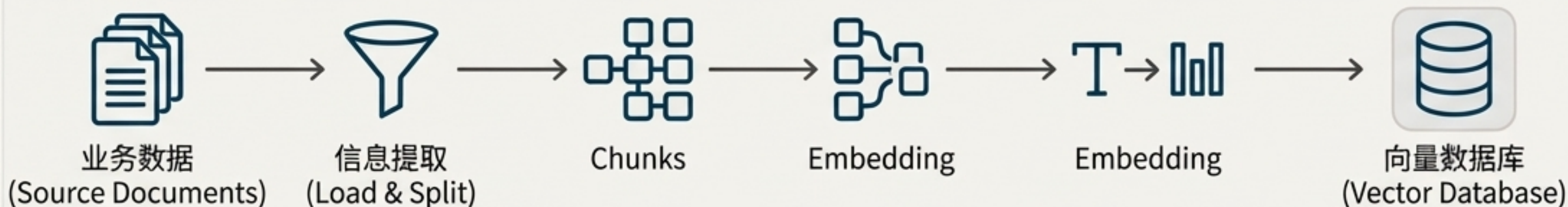
Assistant Response Area

根据您提供的信息，2023-2024赛季的英超冠军是曼城队。

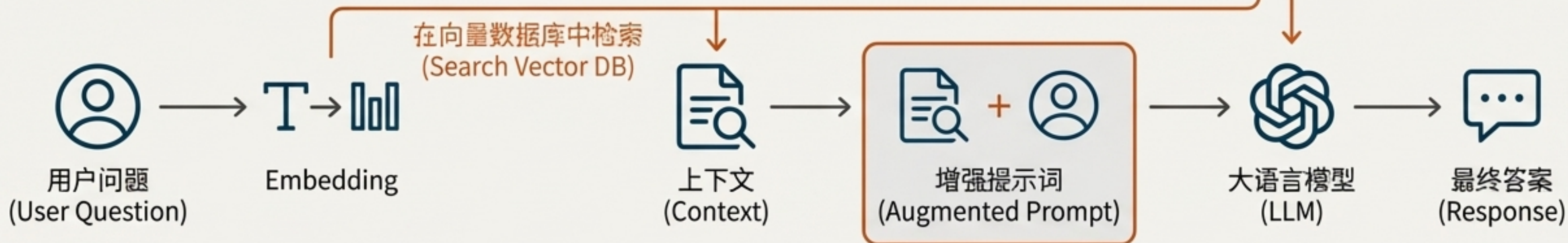
只要在提问时提供正确的“上下文”，大模型就能利用其强大的推理能力，给出精准答案。这就是 RAG 的核心思想。

解决方案蓝图：检索增强生成（RAG）架构

索引流程（离线） - Indexing Pipeline (Offline)



推理流程（在线） - Inference Pipeline (Online)



RAG 将知识存储与大模型的推理能力解耦，创建了一个灵活、可扩展且能实时更新的知识系统。

实践第一步：数据准备 - 构建专属知识库

1. 加载文档

```
from langchain_community.document_loaders import
    DirectoryLoader
loader = DirectoryLoader('.../docs/', glob="*.txt")
documents = loader.load()
```

2. 切分文档

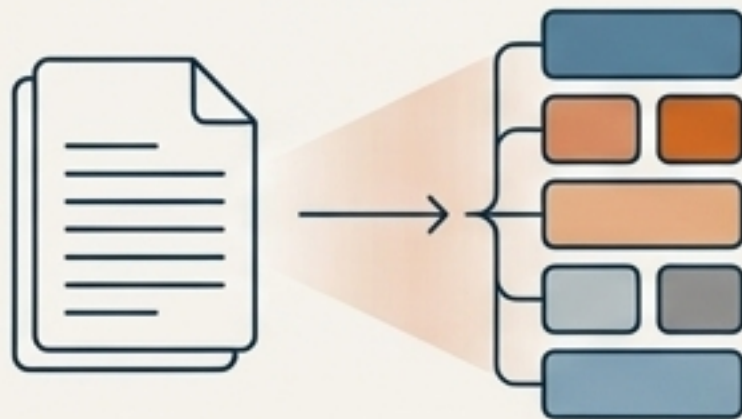
```
from langchain.text_splitter import CharacterTextSplitter
text_splitter = CharacterTextSplitter(
    chunk_size=1024,
    chunk_overlap=128
)
chunks = text_splitter.split_documents(documents)
```

加载 (Load):

从不同来源（如文件夹、数据库）批量读取原始文档。我们在这里使用 `DirectoryLoader` 加载所有 `.txt` 文件。

分割 (Split):

将长文档切分为更小的、有意义的文本块 (Chunks)。这是确保后续检索相关性的关键。`chunk\_size` 定义了每个块的最大长度，`chunk\_overlap` 允许块之间有内容重叠，以保持语义连续性。



实践第二步：嵌入与存储 - 为知识建立智能索引

```
# 3. 嵌入与存储
from langchain_openai import OpenAIEmbeddings
from langchain_weaviate.vectorstores import Weaviate
import weaviate

# 连接 Weaviate 客户端
client = weaviate.Client(...)

# 定义 Embedding 模型
embeddings = OpenAIEmbeddings(openai_api_key="...")

# 创建 Vector Store 并存入数据
vectorstore = Weaviate.from_documents(
    client=client,
    documents=chunks,
    embedding=embeddings
)
```

嵌入 (Embed) :

使用 `OpenAIEmbeddings` 模型将每个文本块转换为一个高维数学向量。这些向量能够捕捉文本的深层语义含义。

存储 (Store) :

将文本块及其对应的向量存入专门的向量数据库 (Vector Database) , 如 `Weaviate`。该数据库为高效的相似性搜索进行了优化。

这个过程就像是为我们的知识库创建一个超级智能的“卡片目录”，它能理解词语背后的真实含义。

实践第三步：检索与生成 - 让知识流动起来

```
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
from langchain.prompts import ChatPromptTemplate

# 1. 创建检索器
retriever = vectorstore.as_retriever()

# 2. 定义提示词模板
template = """
你是一个问答机器人助手。使用以下检索到的上下文来回答问题。
如果你不知道答案，就说你不知道。
上下文: {context}
问题: {question}
"""
prompt = ChatPromptTemplate.from_template(template)

# 3. 构建 RAG 链
rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | llm # ChatOpenAI model defined earlier
    | StrOutputParser()
)
```

检索 (Retrieve) :

当用户提问时，**retriever** 会将问题向量化，并在向量数据库中找到语义最相关的知识块。

增强 (Augment) :

LCEL 管道 (**|**) 自动将检索到的 **context** 和用户的 **question** 填充到 **prompt** 模板中。

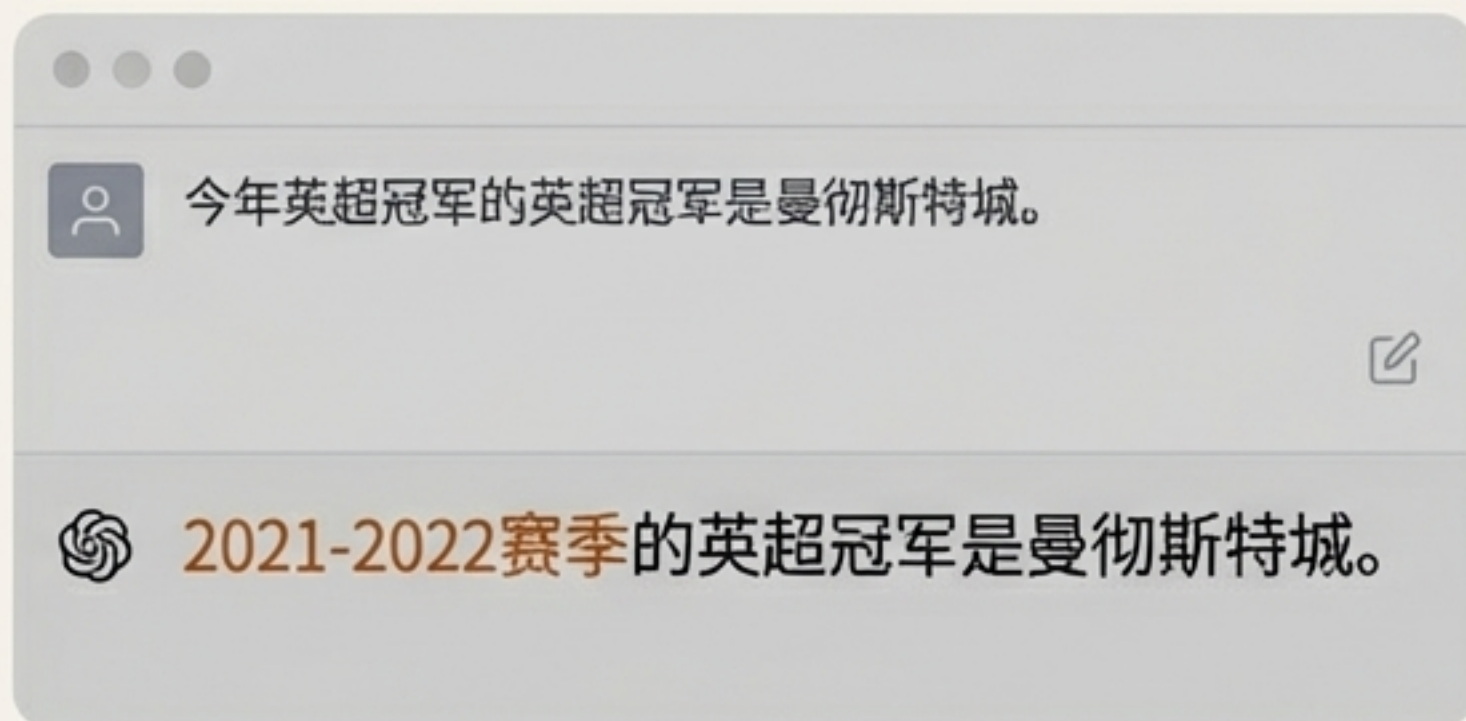
生成 (Generate) :

最终，包含丰富上下文的提示词被发送给 **llm**，生成精准、基于事实的答案。

成果检验：从回答错误到精准作答

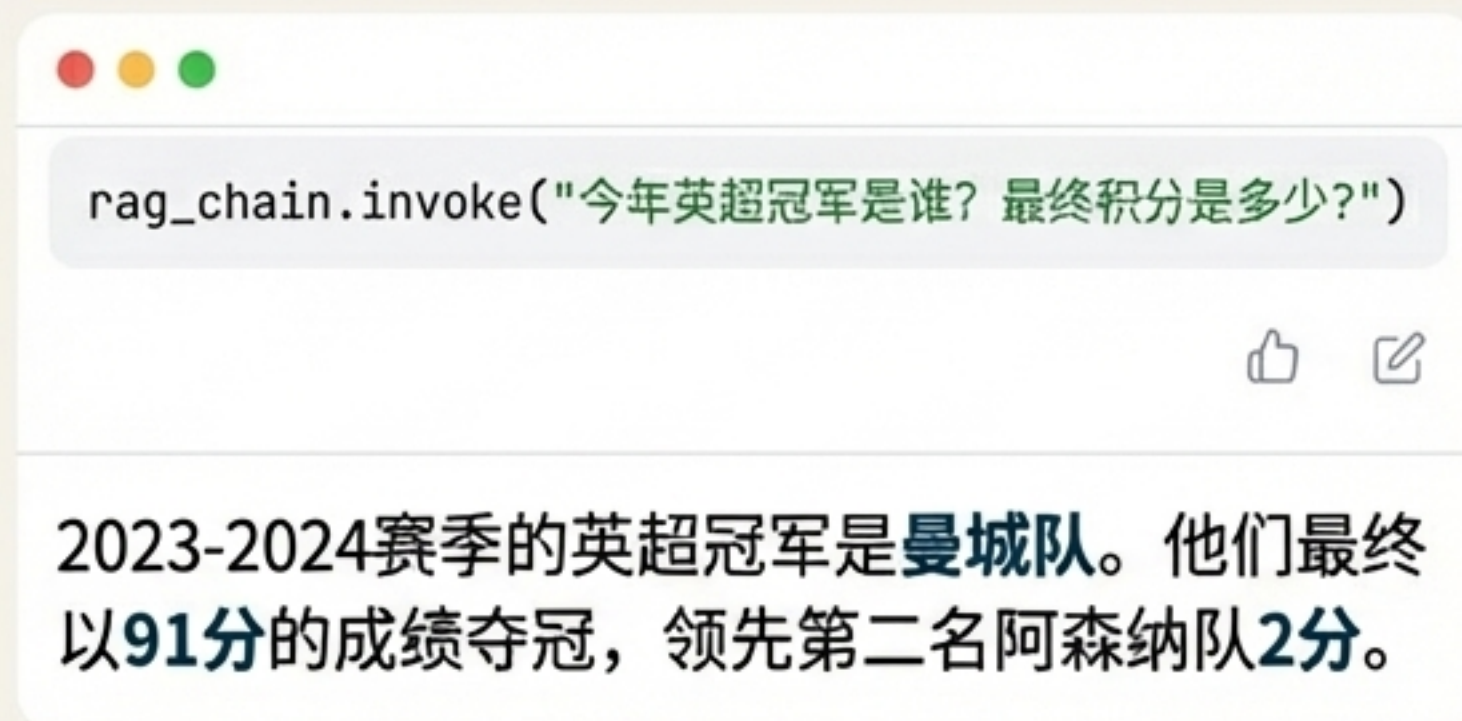
之前 (Before)

标准大模型 (Standard LLM)



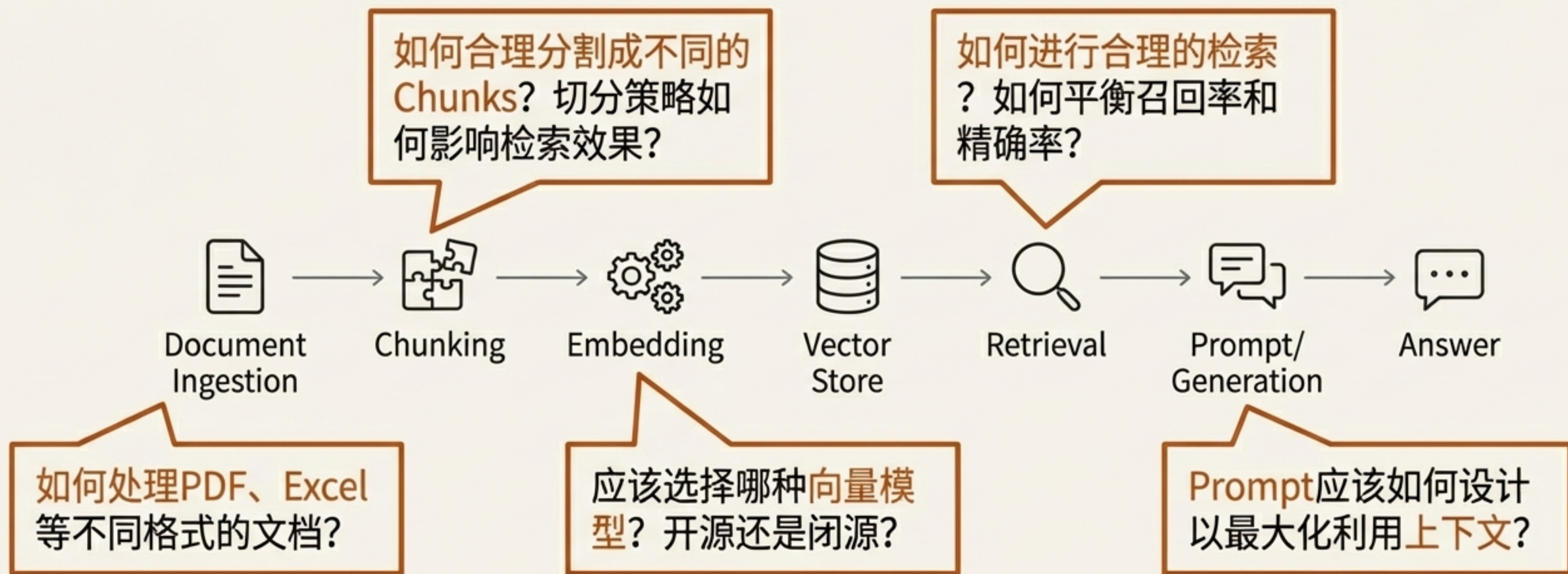
之后 (After)

RAG 增强后 (After RAG Augmentation)



通过连接外部知识库, 我们的 RAG 系统成功克服了模型的知识局限性, 提供了准确、详细且可验证的答案。

真实世界：RAG 实践中的核心工程挑战



搭建一个原型很简单，但构建一个生产级的 RAG 系统是一个持续优化和评估的系统工程。

优化流水线：应对挑战的工具与策略

挑战：信息提取与分块



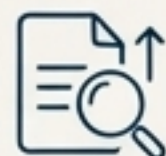
策略与工具（Strategies & Tools）：使用 PyMuPDF, pandas 等库处理特定格式；采用基于 NLTK、spaCy 的句法分析或基于逻辑结构（如标题、段落）的智能分块策略。

挑战：向量模型选择



策略与工具（Strategies & Tools）：根据任务领域选择最优模型(如 BERT、RoBERTa 系列)；考虑 MTEB 等排行榜进行评测；使用 FAISS、Milvus、Pinecone 等成熟的向量数据库。

挑战：检索质量优化



策略与工具（Strategies & Tools）：实施混合搜索（向量+关键词）；在检索后加入 Re-ranking 模型进行二次排序；优化检索数量 top_k。

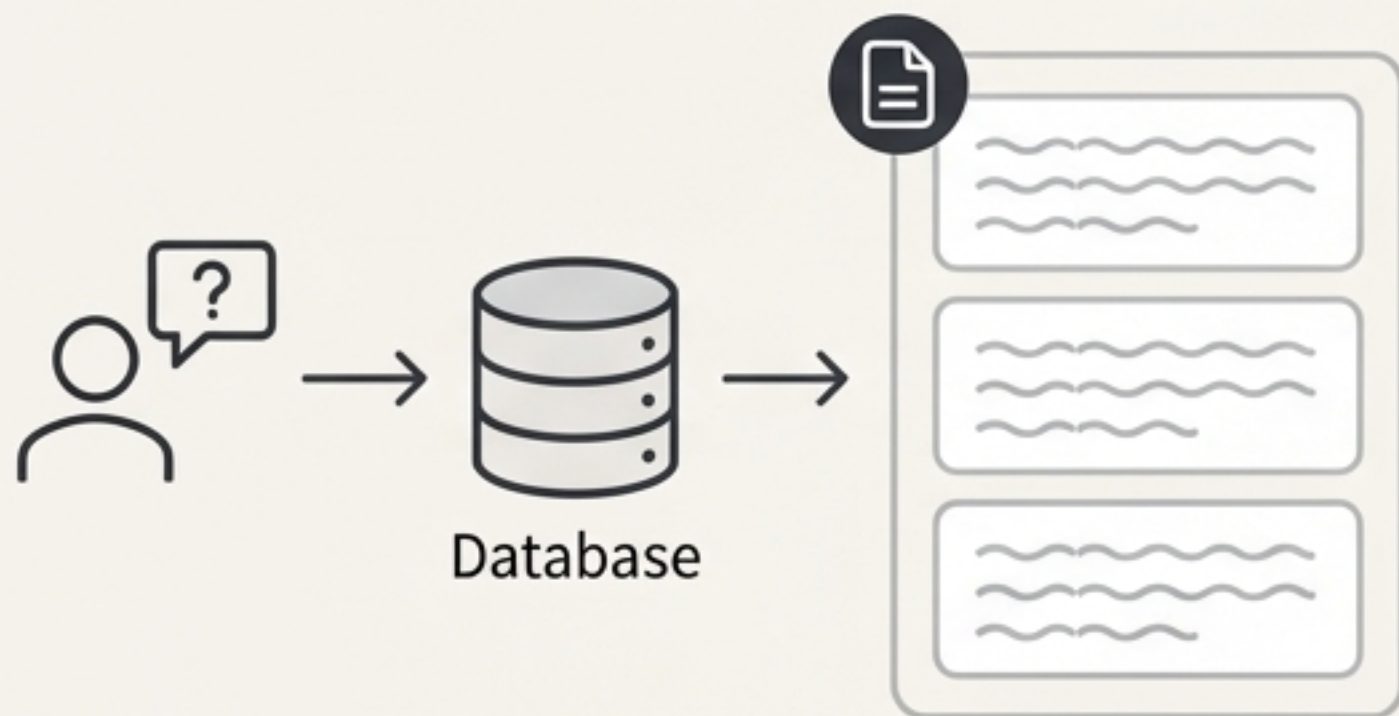
挑战：提示词与生成



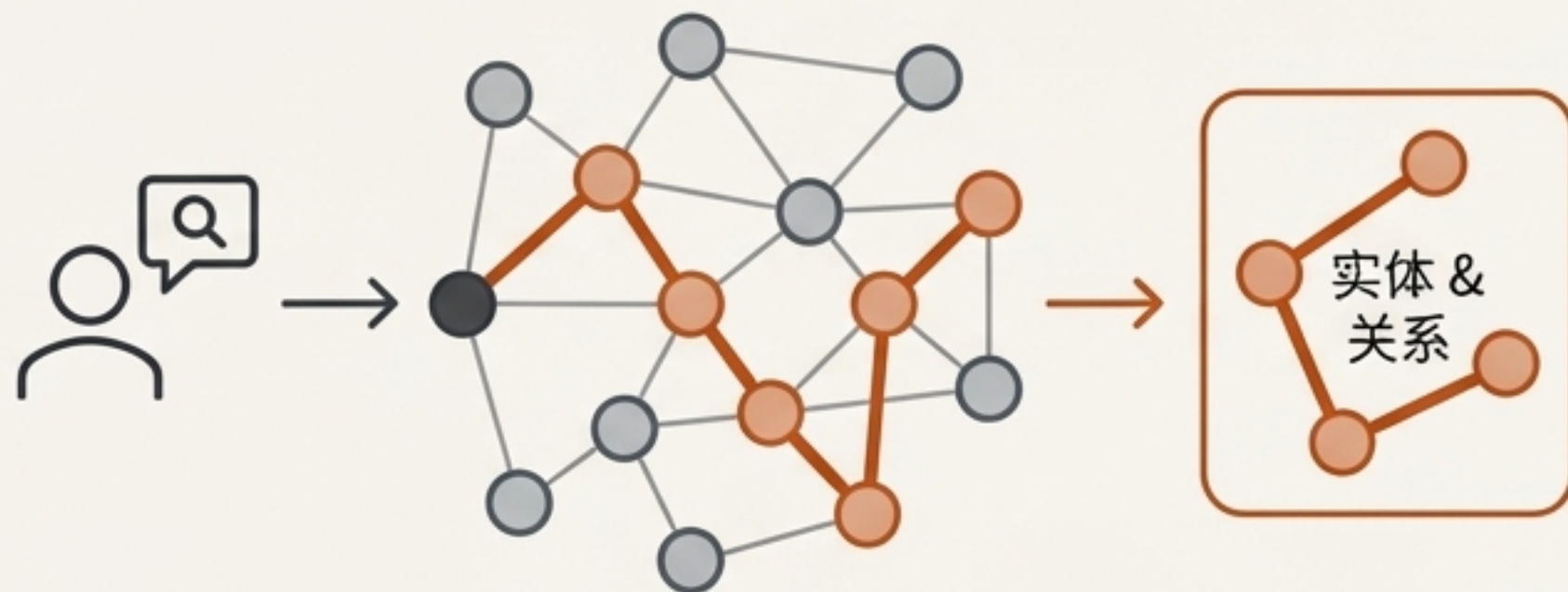
策略与工具（Strategies & Tools）：应用高级提示词工程（Prompt Engineering）技术；对生成结果进行二次处理，如格式化、内容补充和风格调整。

未来前沿：超越简单的向量检索 - GraphRAG

传统 RAG (Traditional RAG)



GraphRAG



先进的方法，如 GraphRAG，不再将文档视为独立的文本块，而是从中提取实体和关系，构建成一个知识图谱。这使得系统能够通过检索相互关联的概念来进行更复杂的、多跳的推理，而不仅是找到相似的文本片段。

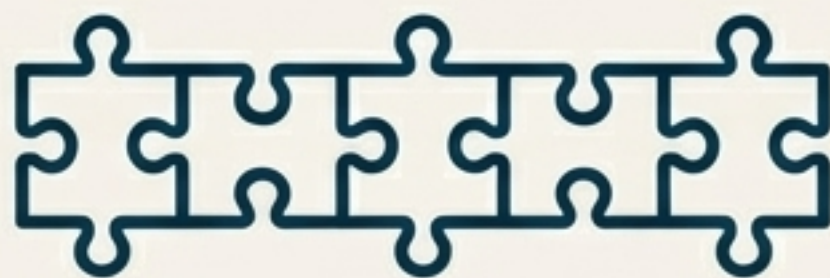
RAG 的未来在于更精细化的知识表示和更智能的检索策略。

核心总结：你的 RAG 行动指南



问题导向 (Problem-Oriented):

RAG 精准解决了大模型在知识时效性、私有性和可信度方面的核心痛点，是构建可靠 AI 应用的基石。



模块化架构 (Modular Architecture):

“加载-分割-嵌入-检索-生成”的流水线是构建任何 RAG 系统的强大心智模型，理解并优化每个模块是成功的关键。



工程化思维 (Engineering Discipline):

RAG 的成功本质上是一个工程挑战。专注于对每个环节进行迭代优化和严格评估，尤其要重视检索质量，才能获得最佳的系统性能。